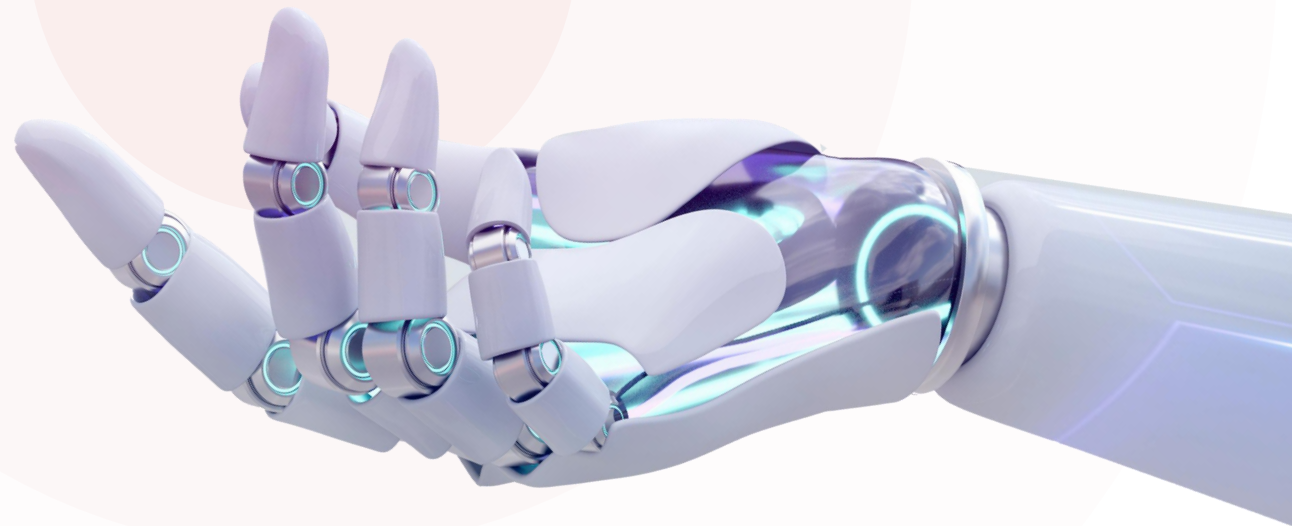


Python核心语法

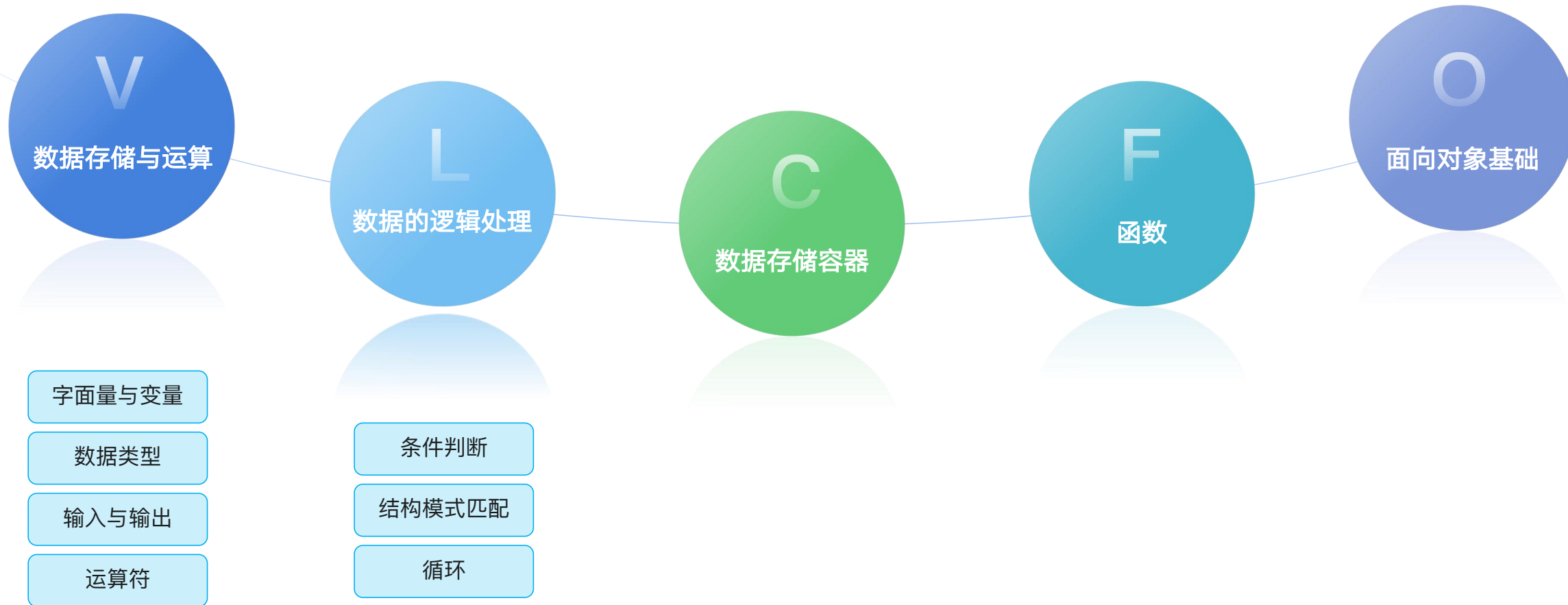
AI Copilot



多一句没有，少一句不行，用更短的时间，教会更实用的技术



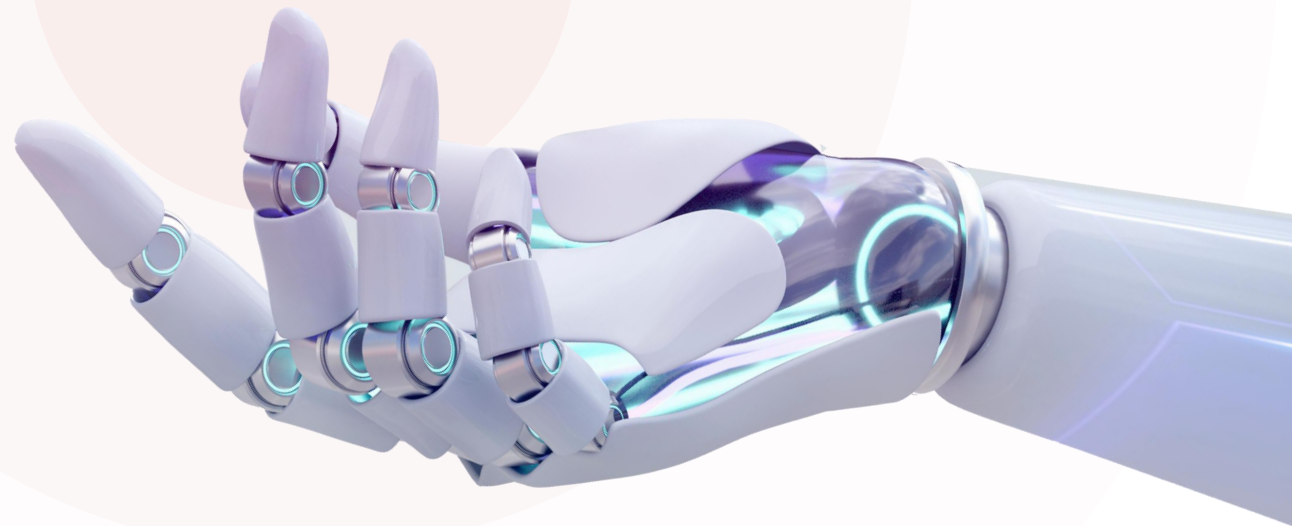
Python核心语法



Python核心语法

—— 数据存储容器 ——

AI Copilot



多一句没有，少一句不行，用更短的时间，教会更实用的技术

```
# 定义变量  
score = 695
```

```
# 定义变量  
score1 = 695  
score2 = 558  
score3 = 622  
score4 = 589  
score5 = 645
```

```
# 定义变量  
score1 = 695  
score2 = 558  
score3 = 622  
score4 = 589  
score5 = 645  
score6 = 607  
score7 = 577  
score8 = 552  
score9 = 475  
score10 = 602
```

.....

繁琐

```
# 定义变量
score1 = 695
score2 = 558
score3 = 622
score4 = 589
score5 = 645
score6 = 607
score7 = 577
score8 = 552
score9 = 475
score10 = 602
.....
```

繁琐

数据容器

```
# 定义变量
score_list = [695, 558, 622, 589, 645, 607, 577, 552, 475, 602, ...]
```

简洁、优雅

数据容器

- 一种可以容纳多份数据的数据类型（容器），容纳的每一份数据称之为1个元素，每一个元素都可以是任意类型的数据，如：字符串、数字、布尔等。



是否支持重复元素

是否可以修改

是否有序

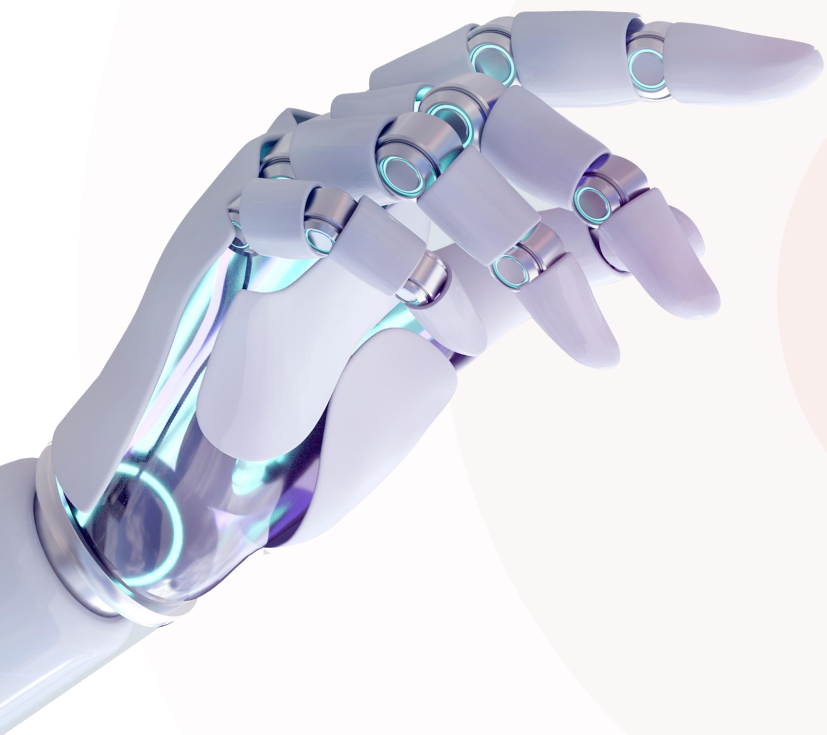
列表 (list)

字符串 (str)

元组 (tuple)

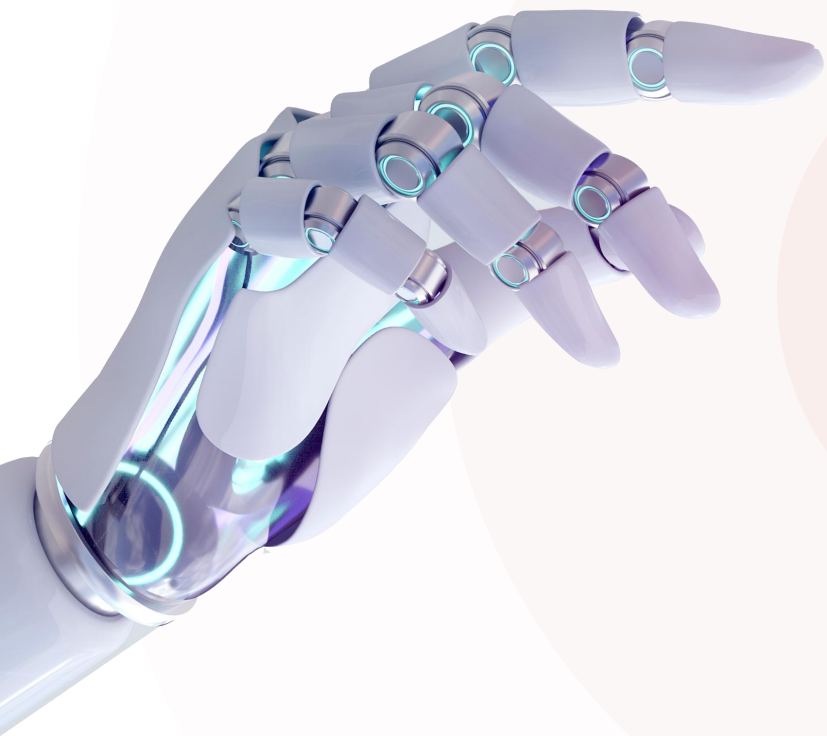
集合 (set)

字典 (dict)



CONTENT 目录

- ◆ 列表 (list)
- ◆ 字符串 (str)
- ◆ 元组 (tuple)
- ◆ 集合 (set)
- ◆ 字典 (dict)
- ◆ 综合案例



1

列表(list)

- ◆ 介绍
- ◆ 常见方法
- ◆ 案例

列表-介绍

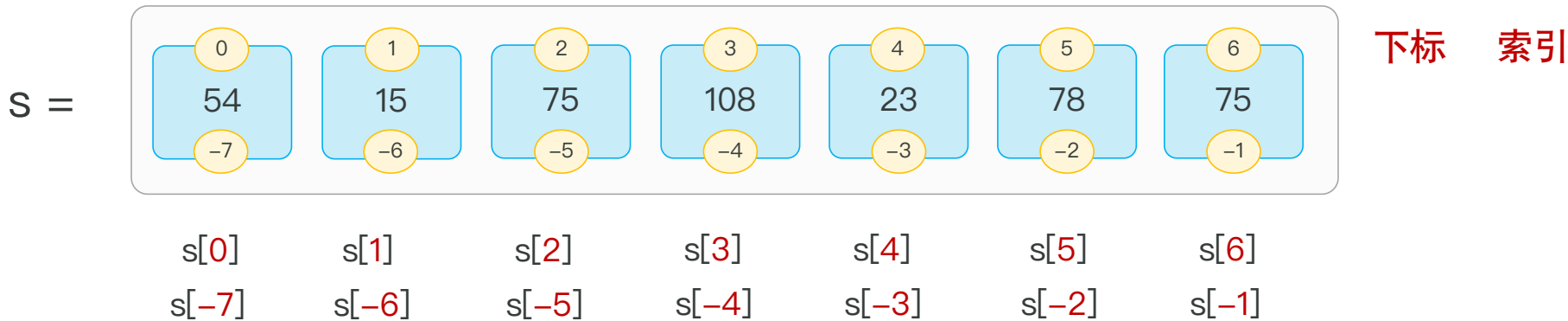
- 列表是数据容器中的一类，是一次性可以存储多个数据（元素）的。

- 定义： 列表名称 = [元素1, 元素2, 元素3, 元素4, 元素5...]

```
s = [54, 15, 75, 108, 23, 78, 75]
```

- 特点：

- 可以存储不同类型的元素
- 元素有序、可以重复、元素可以修改



序列类型：容器中的元素按特定顺序排列的、可通过索引访问的容器类型称之为序列。

注意：从前向后（正向索引），下标从0开始；从后向前（反向索引），下标从-1开始。

小结

1. 列表的定义方式及特点

- 列表名 = [元素1, 元素2, 元素3]
- 可以存放不同类型元素、可重复、有序、元素可以修改

2. 列表的索引

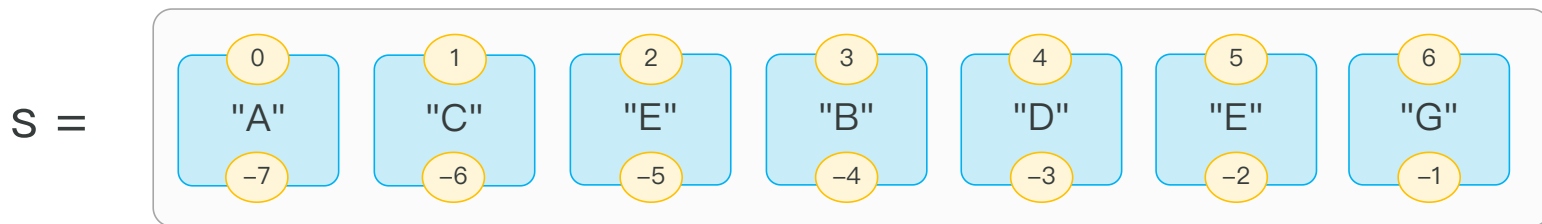
- 正向索引：从0开始
- 反向索引：从-1开始

3. 列表元素的查看、修改、删除

- 查看：list1[0]
- 修改：list1[0] = "A"
- 删除：del list1[3]
- 注意：如果指定的索引值超出范围，将会报错

列表-切片

- 介绍：切片是指对操作的数据截取其中一部分的操作。列表、字符串、元组都支持切片操作。

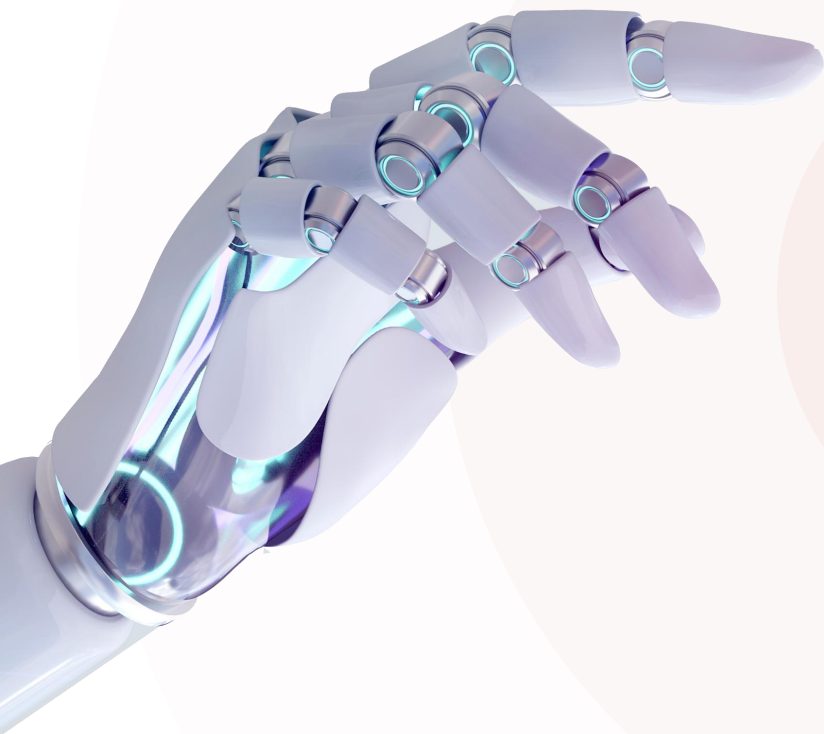


- 语法：序列数据[开始索引:结束索引:步长]
 - 不包含结束索引位置对应的元素(开始索引未指定默认为0；结束索引未指定默认为列表长度(直到列表末尾)；步长未指定默认为1)
 - 索引采用正向、反向索引都可以
 - 步长是选取间隔，默认步长为1
- 举例：
 - s[0:5:1] 切片后的结果为 ["A", "C", "E", "B", "D"]
 - S[0:5:2] 切片后的结果为 ["A", "E", "D"]

BRIEF 小结

1. 列表的切片

- 语法：s[start:end:step]
- 特点：
 - start：开始索引，不指定默认为0（第一个元素的索引）
 - end：结束索引，不指定默认为列表长度(直到列表末尾)
 - step：步长，不指定默认为1



1

列表(list)

- ◆ 介绍
- ◆ 常见方法
- ◆ 案例

什么是方法？

- 方法就像是物品具备的能力或功能。



对象



打电话

方法



发短信

方法



拍照

方法



对象



制冷

方法



制热

方法



通风

方法

True

"Hello"

'A'

80

3.14

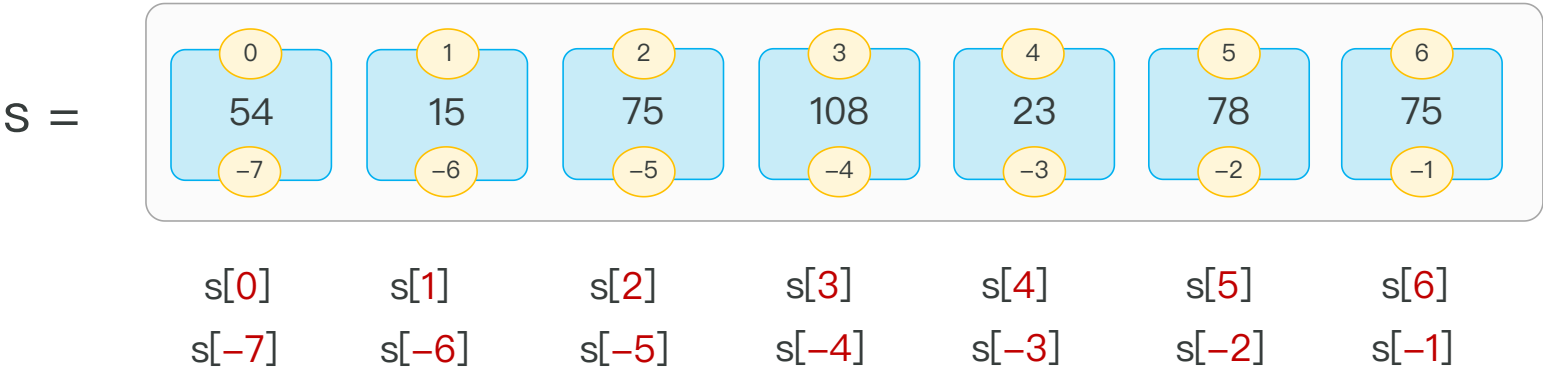
None

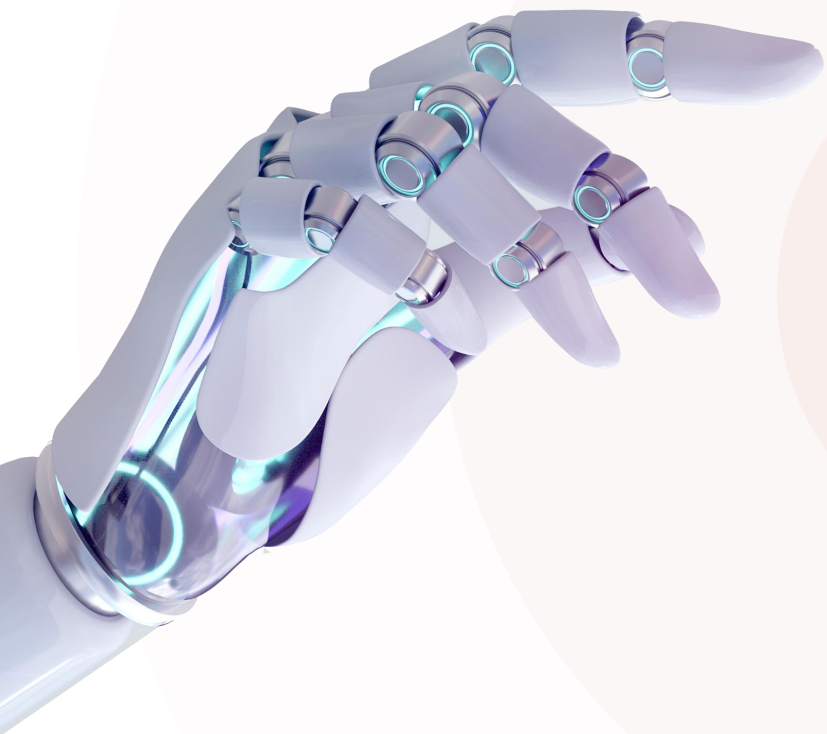
列表的常用方法就是指列表这种数据类型内置的常见功能（添加元素、删除元素、排序等）。

列表-常见方法

● 列表的常见方法：

方法	作用	样例
append()	在列表的尾部追加元素	s.append(10086)
insert()	在指定索引之前，插入该元素	s.insert(0, 92)
remove()	移除列表中第一个匹配到的值	s.remove(75)
pop()	删除列表中指定索引位置的元素（如果未指定索引，默认删最后一个）	s.pop(2) / s.pop()
sort()	对列表进行排序（列表元素的数据类型一致，才可以进行排序）	s.sort()
reverse()	反转列表元素	s.reverse()





1

列表(list)

- ◆ 介绍
- ◆ 常见方法
- ◆ 案例



案例

完成如下需求



1. 将用户输入的10个数字，存储到一个列表中，并将列表中的数字进行排序，输出其中的最小值、最大值和 平均值。

小结

- Python中用于数据统计的常见语句
 - `min()`: 获取最小值
 - `max()`: 获取最大值
 - `sum()`: 求和
 - `len()`: 获取元素的个数



案例

完成如下需求



1. 合并两个列表中的元素，并对合并的结果进行去重处理(去除列表中的重复元素)。

```
# 定义列表
```

```
num_list1 = [19, 23, 54, 64, 875, 20, 109, 232, 123, 54]
```

```
num_list2 = [55, 80, 72, 35, 60, 123, 54, 29, 91]
```

小结

1. 如何快速合并两个列表？

- 使用+运算符直接合并
- 使用*进行解包操作(解包指将列表这一类数据容器解开成独立的元素)

```
num_list1 = [19, 23, 54, 64, 875, 20, 109, 232, 123, 54]
num_list2 = [55, 80, 72, 35, 60, 123, 54, 29, 91]

num_list = num_list1 + num_list2
num_list = [*num_list1, *num_list2]
```

2. 如何判断一个元素是否存在于列表之中？

- 可以使用in运算符，语法为：元素 in 列表
- 返回结果为布尔值，True表示存在，False表示不存在

```
new_list = []
for num in num_list:
    if num not in new_list:
        new_list.append(num)
```



案例

完成如下需求



1. 生成1-20的平方列表。
2. 从如下数字列表中提取所有偶数，并计算其平方，组成一个新的列表。

```
# 定义列表
```

```
num_list = [19, 23, 54, 64, 87, 20, 109, 232, 123, 43, 26, 55, 72]
```

BRIEF 小结

1. 列表推导式

- 含义：列表推导式就是按照一定规则快速生成一个列表的方法
- 格式1：列表名称=[要插入列表的数据 `for i in` 列表]
- 格式2：列表名称=[要插入列表的数据 `for i in` 列表 `if` 条件]



练习

完成如下需求



1. 将如下多个列表合并为一个列表，并去重重复元素，排好序（升序）后输出到控制台。

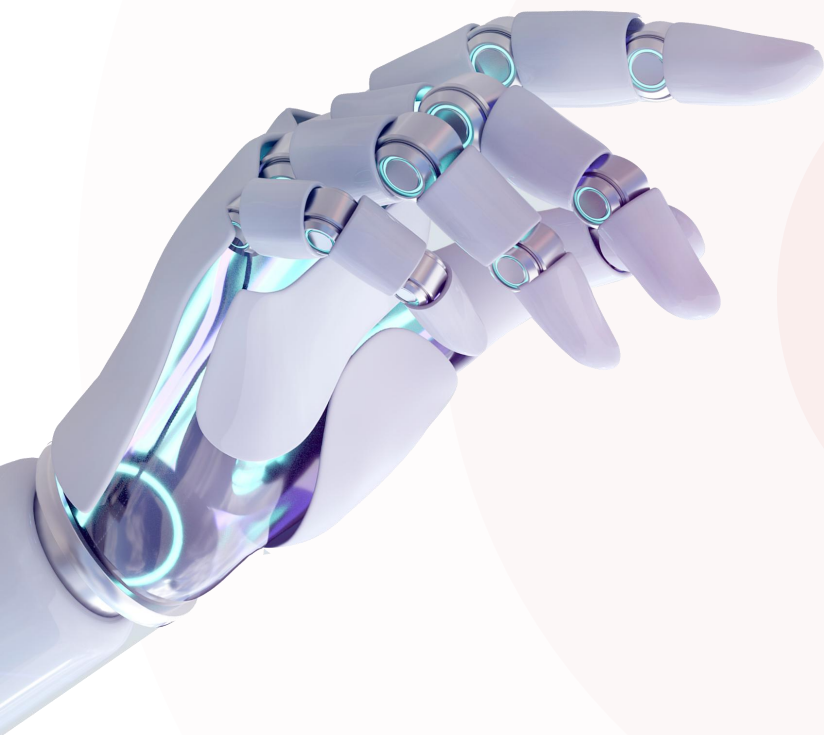
```
# 合并如下三个列表，并对合并后的列表进行元素的去重，然后排好序后输出到控制台
list1 = ['M', 'A', 'C', 'E', 'F', 'G', 'H', 'L', 'N', 'I', 'J', 'K', 'O']
list2 = ['X', 'Z', 'T', 'Y', 'D', 'E', 'F', 'G']
list3 = ['W', 'A', 'S', 'D']
```

2. 将如下列表中能被3 或 5整除的元素提出来，并获取这些数字对应的平方，组成一个新的列表。

```
# 将如下列表中能被3 或 5整除的元素提出来，并获取这些数字对应的平方，组成一个新的列表。
list1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]
```

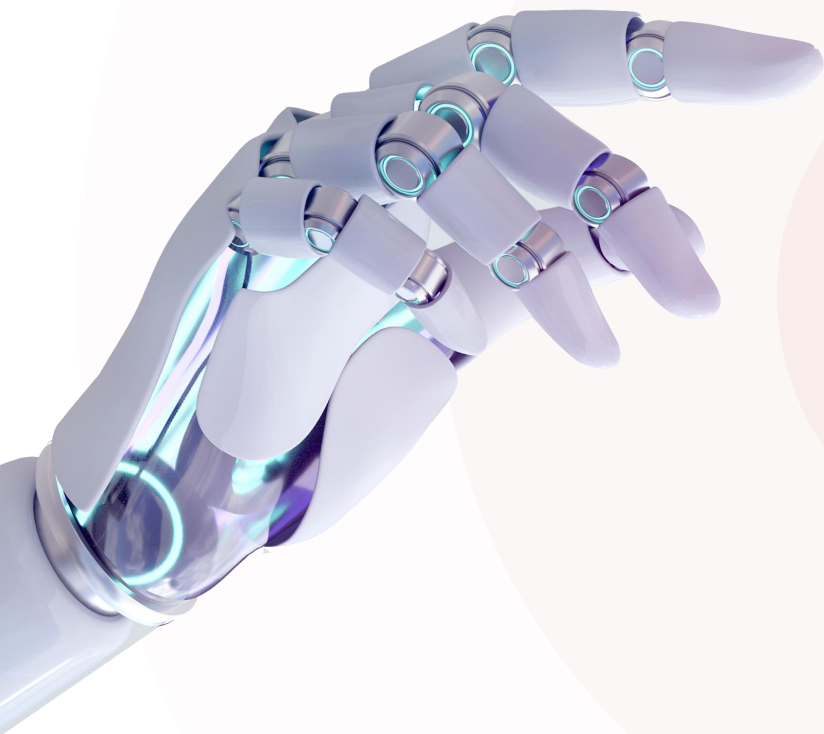
3. 将如下列表中的正数提取出来，封装为一个新的列表。

```
# 将如下列表中的正数提取出来，封装为一个新的列表。
list1 = [11, 2, 31, 4, -5, 15, 17, 28, 49, 10, -11, 16, 54, -14, 36, -16, 87, -39]
```



CONTENT 目录

- ◆ 列表 (list)
- ◆ 字符串 (str)
- ◆ 元组 (tuple)
- ◆ 集合 (set)
- ◆ 字典 (dict)
- ◆ 综合案例



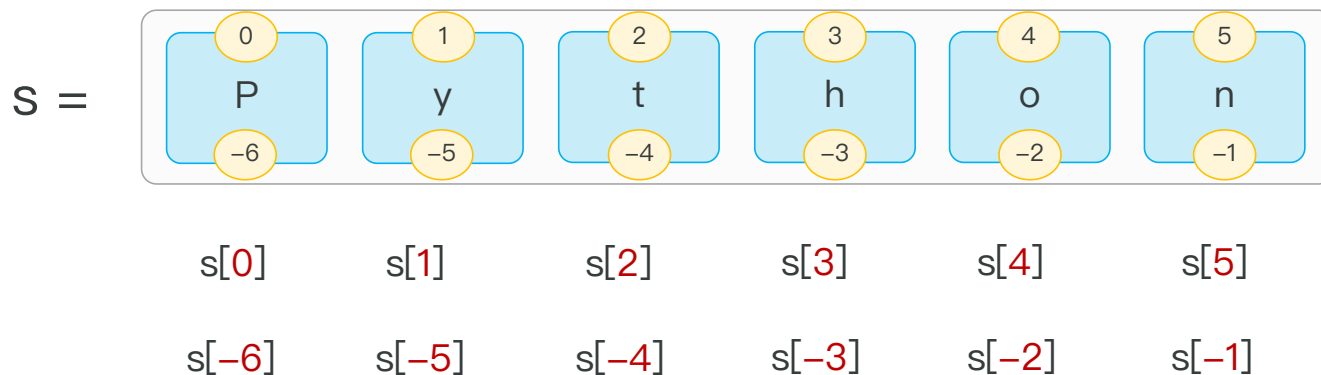
2

字符串

- ◆ 基本操作
- ◆ 常用方法
- ◆ 案例

字符串

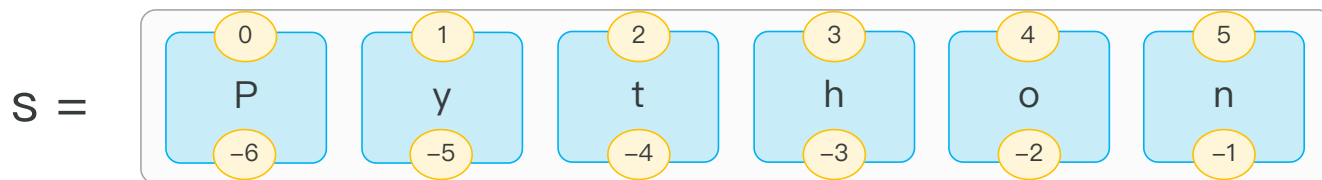
- 介绍：字符串是字符的容器，一个字符串中可以存放任意数量的字符。如：`"Python"` `'Python'` `"""Python"""`
- 特点：不可变性（无法修改）、有序性、可迭代性。
- 字符串中的每一个字符元素都有其对应的下标（索引），通过元素对应的索引，就可以获取到对应的元素。



注意：从前向后（正向索引），下标从0开始；从后向前（反向索引），下标从-1开始。

字符串-切片

- 介绍：切片是指对操作的对象截取其中一部分的操作。字符串、列表、元组都支持切片操作。



- 语法：序列对象[开始索引:结束索引:步长]
 - 不包含结束索引位置对应的元素(开始索引未指定默认为0；结束索引未指定默认为-1；步长未指定默认为1)
 - 索引采用正向、反向索引都可以
 - 步长是选取间隔，默认步长为1 (如果是-1, 表示从后向前)
- 举例：
 - s[0:5:1] 切片后的结果为 "Pytho"
 - S[0:5:2] 切片后的结果为 "Pto"

小结

1. 字符串的特点

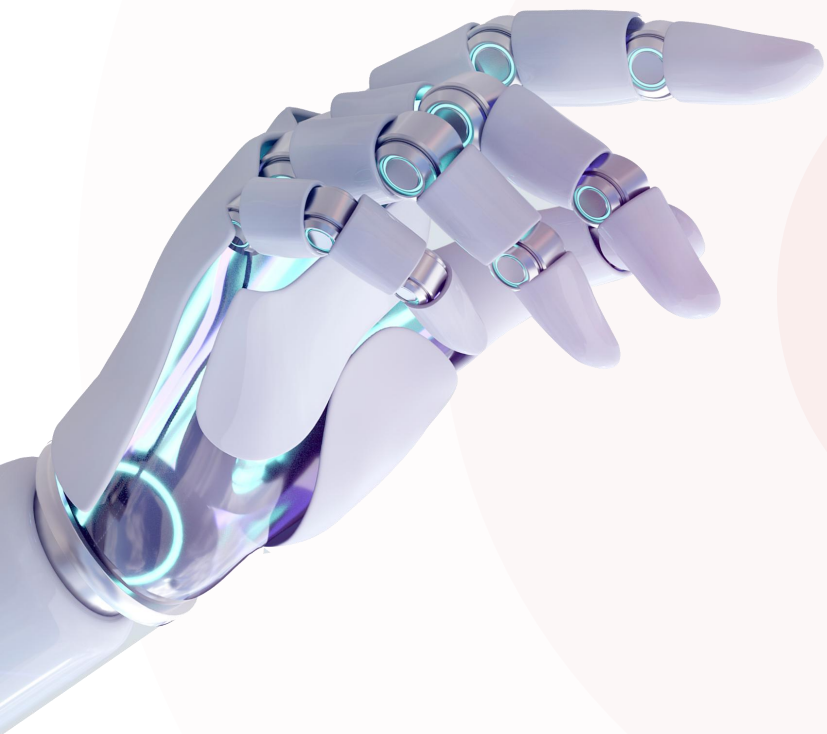
- 不可变性（无法修改）、有序性、可迭代性

2. 字符串的索引

- 正向索引：从0开始
- 反向索引：从-1开始

3. 字符串的切片

- 语法：s[start:end:step]
- 特点：
 - start：开始索引，不指定默认为0（第一个元素的索引）
 - end：结束索引，不指定默认为-1（最后一个元素的索引）
 - step：步长，不指定默认为1（-1表示从后向前）



2

字符串

- ◆ 基本操作
- ◆ 常用方法
- ◆ 案例



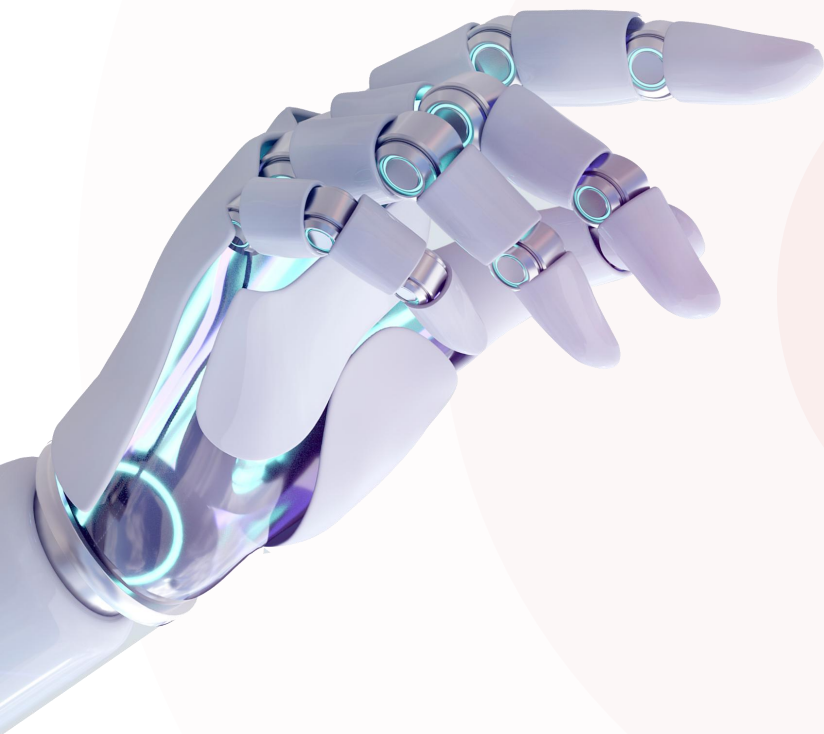
字符串-常用方法

● 字符串操作常用方法：

方法	作用	样例
find()	在字符串中查找子串，返回第一次出现的索引位置，找不到返回-1	s.find('Python')
count()	统计子串在字符串中出现的次数	s.count('H')
upper()	将字符串中的所有字母转换为大写	s.upper()
lower()	将字符串中的所有字母转换为小写	s.lower()
split()	将字符串按指定分隔符分割成列表	s.split(' ')
strip()	去除字符串两端的空白字符或指定字符	s.strip() / s.strip('*')
replace()	将字符串中的指定子串替换为新的子串	s.replace('H','C')
startswith()	检查字符串是否以指定子串开头，返回布尔值	s.startswith('P')

BRIEF 小结

1. 请描述如下字符串方法的作用？
 - find()、count()、upper()、lower()、split()、strip()、replace()、startswith()



2

字符串

- ◆ 基本操作
- ◆ 常用方法
- ◆ 案例



案例

完成如下需求



1. 邮箱格式验证：用户输入一个邮箱，验证邮箱格式是否正确(包含一个@和至少一个.)，如果输入正确，输出"邮箱格式正确"，否则输出"邮箱格式错误"。

itheima@163.com

itheima@mail.itcast.cn

BRIEF 小结

1. 如何判断指定的子字符串是否出现在字符串之中？
 - in运算符，返回bool值

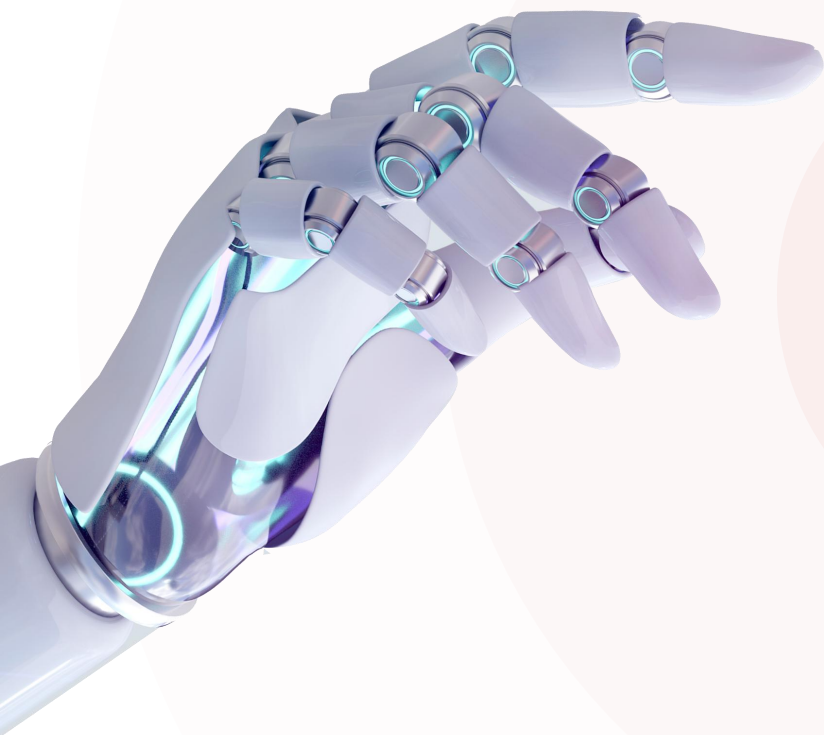


练习

完成如下需求



1. 输入一个字符串, 判断该字符串是否是回文(两边对称)。
 - 黄山落叶松叶落山黄
 - 上海自来水来自海上
2. 将用户输入的10个字符串, 反转后全部转换为大写, 然后记录在列表中, 最后将列表内容, 遍历输出出来。



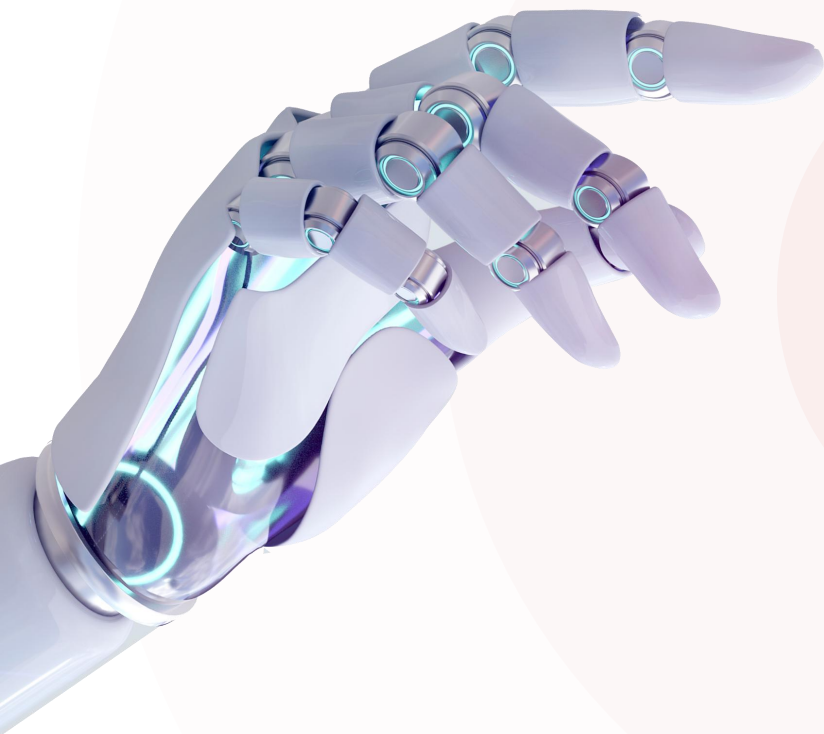
CONTENT 目录

- ◆ 列表 (list)
- ◆ 字符串 (str)
- ◆ 元组 (tuple)
- ◆ 集合 (set)
- ◆ 字典 (dict)
- ◆ 综合案例

为什么要用元组？



- 列表有什么特点？
 - 元素可重复、有序、可以修改的
- 如果要记录一些信息，而这些信息，不能被修改，只能查询呢？
 - 列表就不适合了
 - 此时就可以考虑使用元组，元组与列表最大的不同点在于：**元组一旦定义完成，不可修改**



3

元组(tuple)

- ◆ 基本操作
- ◆ 组包与解包

元组

- 介绍：元组是**不可变**的序列，类似于列表，但创建后不能修改。

- 特点：

- 可以存储不同类型的元素
- 元素可以重复、有序、**不可以修改**（支持索引访问、切片）

- 定义：

```
# 定义元组
元组名称 = (元素1, 元素2, ...)
```



```
# 定义空元组
元组名称 = ()
元组名称 = tuple()
```

```
# 定义元组
t1 = (5, 7, 9, 1, 2, 3, 10, 6, 4, 8, 12, 7, 5)
```



```
# 定义空元组
t2 = ()
t3 = tuple()
```

- 方法：

- count(): 统计某元素在元组中出现的次数
- index(): 查找某个元素在元组中的索引位置(第一次出现的位置)

BRIEF 小结

1. 元组有什么特点？

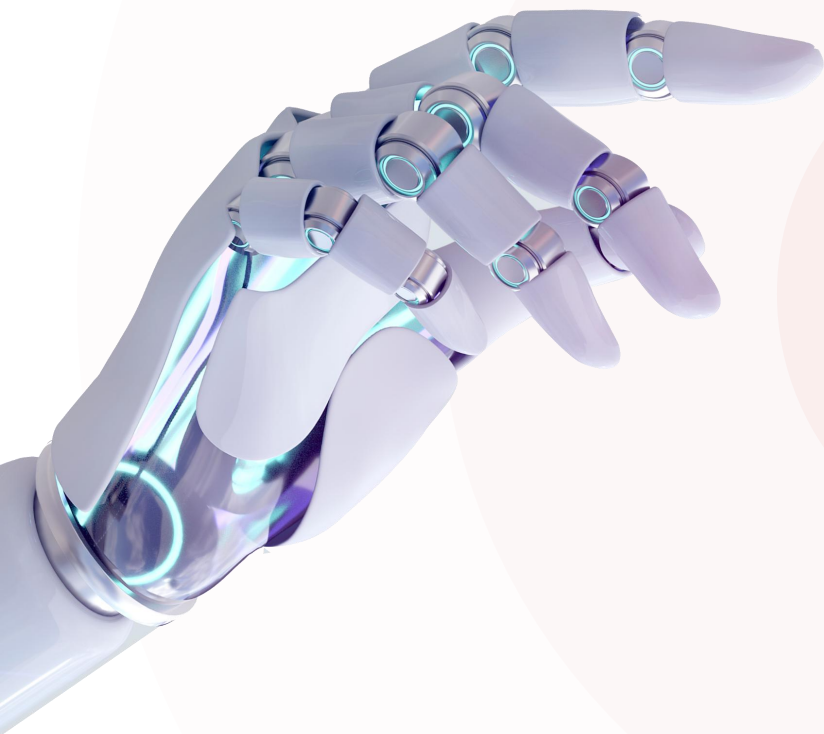
- 元素可以重复
- 有序
- 不可修改（只读）

2. 元组的两个查询方法？

- `count()`：统计指定元素的数量
- `index()`：获取指定元素的索引，如果该元素不存在将报错

3. 元组使用注意事项？

- 定义单元素元组时，需要在结尾加上逗号，比如 `('A',)`



3

元组(tuple)

- ◆ 基本操作
- ◆ 组包与解包

元组-组包与解包

- 组包 (Packing)：将多个值合并到一个容器（元组、列表）中。
- 解包 (Unpacking)：将容器(元组、列表)解开成独立的元素，分别赋值给多个变量。

```
# 定义元组, 组包
```

```
t1 = (5, 7, 9, 1)
```

```
# 定义元组, 组包
```

```
t2 = 5, 7, 9, 1
```

```
# 基础解包
```

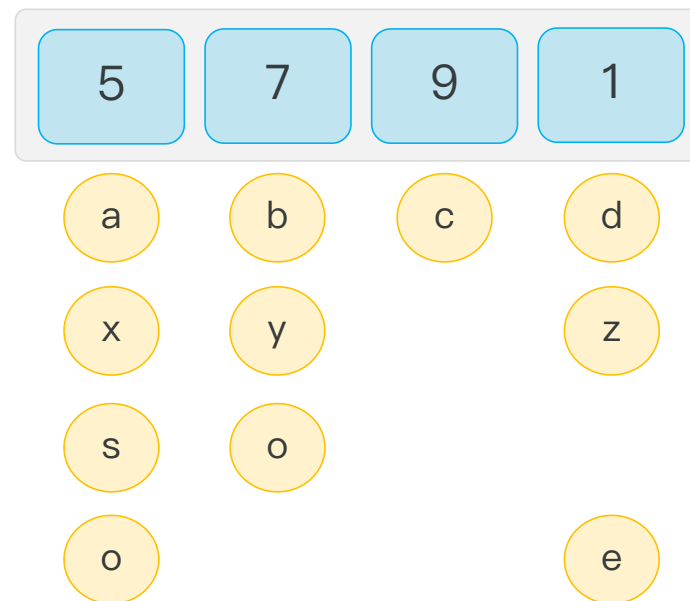
```
a, b, c, d = t1
```

```
# (*)扩展解包
```

```
x, *y, z = t2 # x为5, y为[7, 9], z为1
```

```
s, *o = t2 # s为5, o为[7, 9, 1]
```

```
*o, e = t2 # o为[5, 7, 9], e为1
```



说明：在元组解包时，* 表示收集剩余的所有元素，允许我们处理不确定数量的元素（生成列表，以便于可以进行进一步的处理）。



案例

完成如下案例



1. 现有两个变量，分别为： $a = 10$ ， $b = 20$ ， 现需要将这两个变量值交换，然后输出到控制台。
2. 现有三个变量，分别为： $a = 100$ ， $b = 200$ ， $c = 300$ ， 现需要将这三个变量值进行交换，将 a, b, c 的值分别赋值给 c, a, b ，并将其输出到控制台。

BRIEF 小结

- 组包与解包？

- 组包（Packing）：将多个值合并到一个容器（元组、列表）中。
- 解包（Unpacking）：将容器中的元素分别赋值给多个变量。
 - 在元组解包时，* 表示收集剩余的所有元素，允许我们处理不确定数量的元素

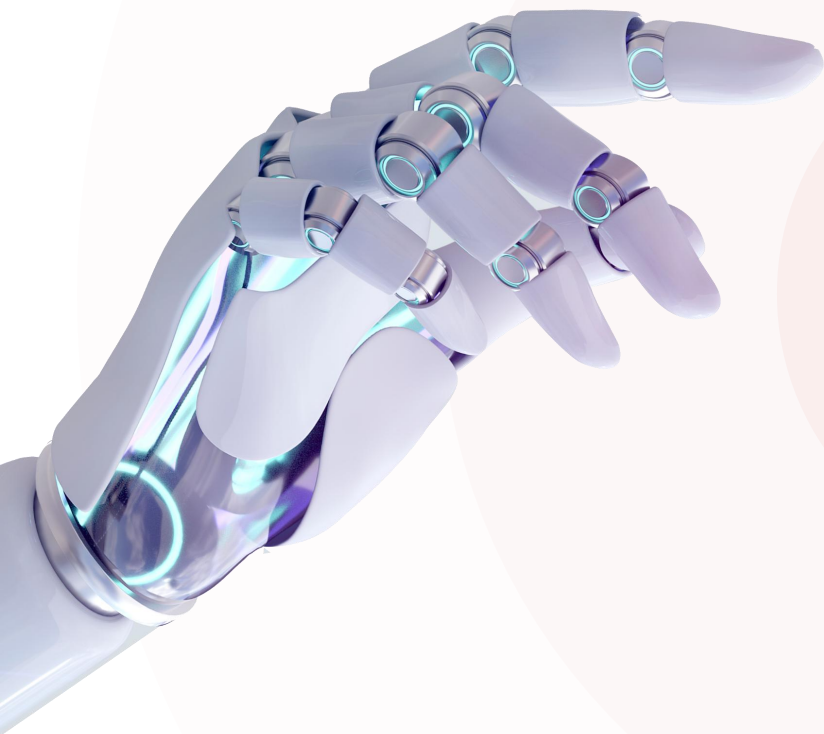
案例

基于已学习的数据容器完成如下需求



- 根据提供的学生成绩单，完成如下需求：
 1. 计算每个学生的总分、各科平均分，然后一并输出出来。
 2. 统计各科成绩的最低分、最高分、平均分，并输出。
 3. 查找成绩优秀（平均分大于90）的学生，并输出。

学号	姓名	语文	数学	英语
S001	王林	85	92	78
S002	李慕婉	92	88	95
S003	十三	78	85	82
S004	曾牛	88	79	91
S005	周轶	95	96	89
S006	王卓	76	82	77
S007	红蝶	89	91	94
S008	徐立国	75	69	82
S009	许木	86	89	98
S010	遁天	66	59	72



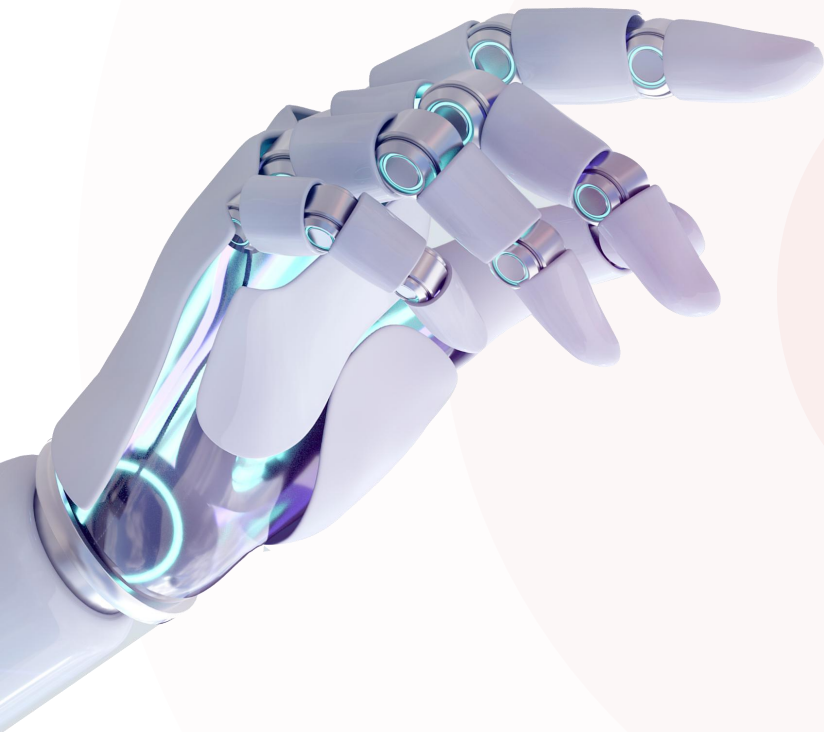
CONTENT 目录

- ◆ 列表 (list)
- ◆ 字符串 (str)
- ◆ 元组 (tuple)
- ◆ 集合 (set)
- ◆ 字典 (dict)
- ◆ 综合案例

为什么要用集合?



- 场景：在业务中，需要定义一个变量，来批量存储用户的手机号（唯一的）？
 - 列表list、元组tuple可以存吗？ 不可以，因为这两种类型是可以存储重复元素的
 - 此时就可以使用集合set来存储，set会自动去重，存储不重复的元素



4

集合(set)

- ◆ 基本操作
- ◆ 案例

集合(set)

- 介绍：集合(set)是一种无序的、不可重复、可修改的数据容器。
- 定义：

```
set集合定义

# 定义集合
s1 = {"C", "D", "X", "T", "O", "U"}

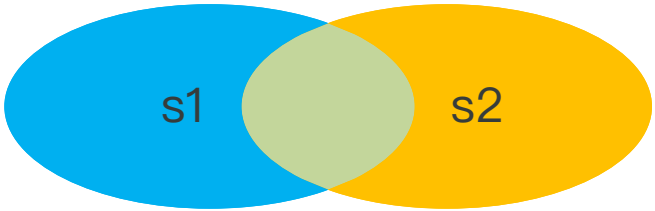
# 定义空集合
s2 = set()
```

注意：空集合的定义不可以使用`{}`，`{}`表示的是空字典；由于集合是无序的，因此是不支持下标索引访问的。

集合(set)-常见操作

- 集合 (set) 中的常见方法：

操作	含义	样例
add(..)	添加元素到集合中	s1.add('t')
remove(..)	移除集合中的指定元素（指定元素不存在将报错）	s1.remove('t')
pop()	随机删除集合中的元素并返回	e = s1.pop()
clear()	清空集合	s1.clear()
difference()	求取两个集合的差集（包含在第一个集合但不包含在第二个集合的元素）	s1.difference(s2)
union()	求取两个集合的并集	s1.union(s2)
intersection()	求取两个集合的交集	s1.intersection(s2)



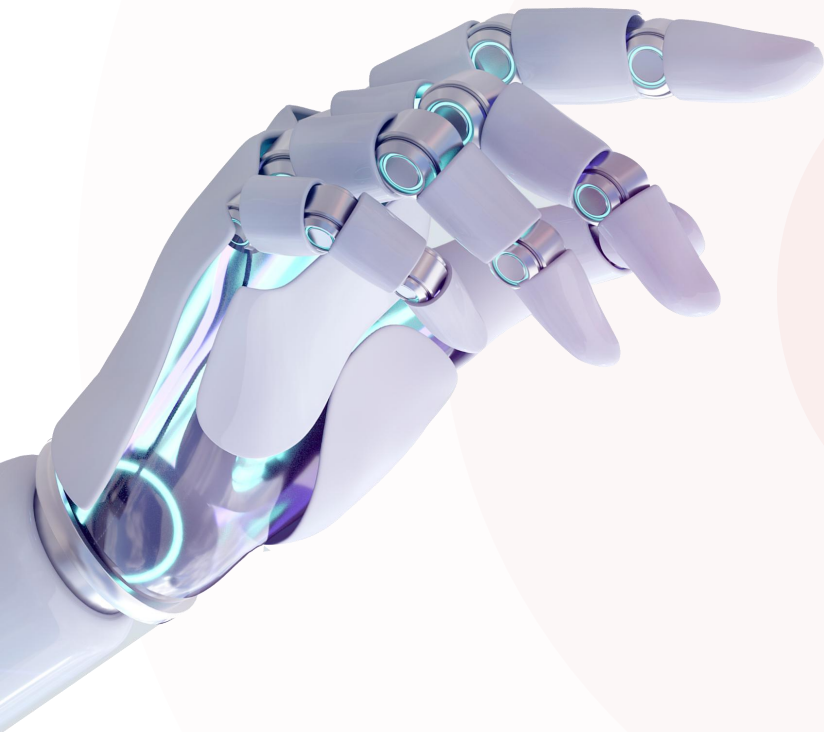
BRIEF 小结

1. 集合(set)的特点?

- 无序、不可重复、可修改 (不要依赖显示顺序)

2. 集合(set)的定义及常用操作

- 定义1: 集合名 = {元素1,元素2,元素3...}
- 定义2: 集合名 = set()
- 常用操作:
 - 添加: s1.add(..)
 - 删除: s1.remove(..) / s1.pop() / s1.clear()
 - 交、并、差集: s1.intersection(s2) / s1.union(s2) / s1.difference(s2)



4

集合(set)

◆ 基本操作

◆ 案例



案例

完成如下需求



- 根据提供的班级学生的选课情况，完成如下需求：
 1. 找出同时选修了法语和艺术的学生
 2. 找出同时选修了所有四门课程的学生
 3. 找出选修了足球，但是没有选修篮球的学生
 4. 统计每一个学生选修的课程数量

```
# 选修足球学生名单
football_set = {"王林", "曾牛", "徐立国", "遁天", "天运子", "韩立", "厉飞雨", "乌丑", "紫灵"}

# 选修篮球学生名单
basketball_set = {"张铁", "墨居仁", "王林", "姜老道", "曾牛", "王蝉", "韩立", "天运子", "李化元", "厉飞雨", "云露"}

# 选修法语学生名单
french_set = {"许木", "王卓", "十三", "虎咆", "姜老道", "天运子", "红蝶", "厉飞雨", "韩立", "曾牛"}

# 选修艺术学生名单
art_set = {"遁天", "天运子", "韩立", "虎咆", "姜老道", "紫灵"}
```

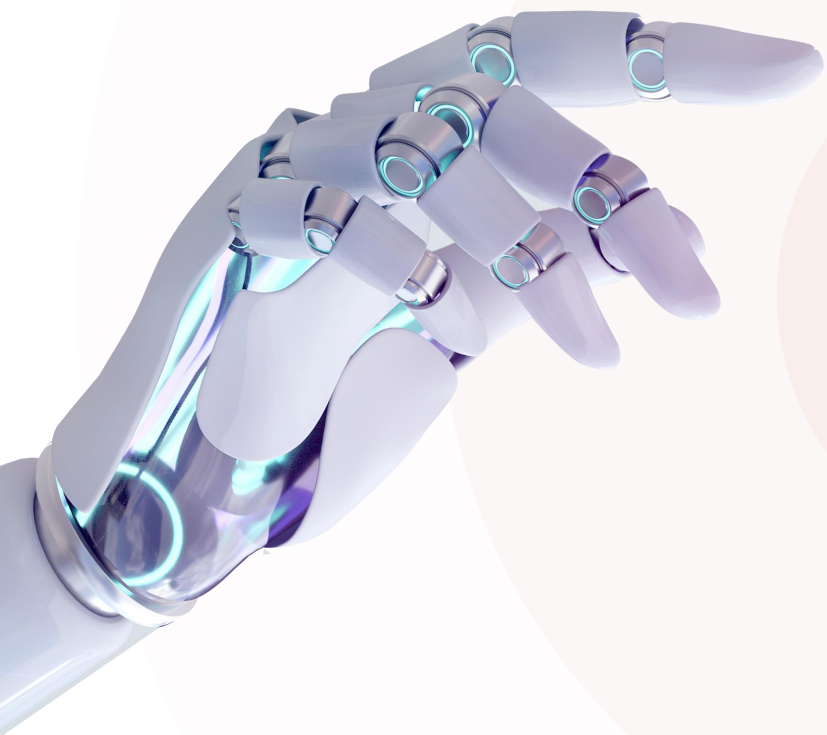
小结

1. 集合set操作的运算符 $\&$ $|$ $-$

- $\&$: 交集
- $|$: 并集
- $-$: 差集

2. 集合推导式的写法

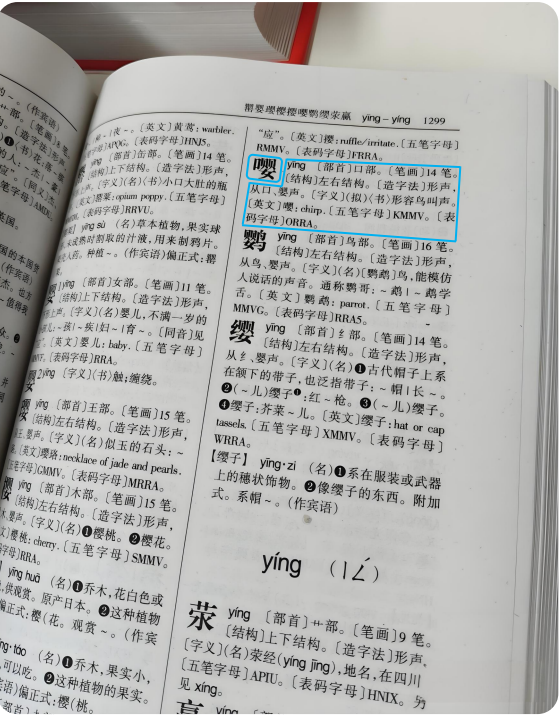
- 变量名称={i表达式 for i in 列表}
- 变量名称={i表达式 for i in 列表 if 条件}



CONTENT 目录

- ◆ 列表 (list)
- ◆ 字符串 (str)
- ◆ 元组 (tuple)
- ◆ 集合 (set)
- ◆ 字典 (dict)
- ◆ 总结

什么是字典？



- 通过【字/词语】，找到该字对应的【解释】。
- 每个【字/词语】对应一个解释，【字/词语】是不能重复的。

dictionary

哈希表 (hash)

映射 (map)

Python中的字典 (dict)，里面存储的是键值对 (key: value) 类型的数据，可以根据键(key)找到对应的值(value)。

为什么要使用字典呢？

- 现需要存储班级学员高考成绩，其中包含 王林(670)，韩立（556），李慕婉（582），紫灵（435），许立国（608）...，该如何存储呢？

```
# 定义列表存储
```

```
name_list = ["王林", "韩立", "李慕婉", "紫灵", "许立国", "王政", "张紫"]
```

```
score_list = [670, 556, 582, 435, 608, 512, 678]
```

繁琐 低效

```
# 定义字典存储
```

```
dict1 = {"王林": 670, "韩立": 556, "李慕婉": 582, "紫灵": 435, "许立国": 608, "王政": 512, "张紫": 678}
```

优雅 高效

5

字典(dict)

- ◆ 字典定义
- ◆ 常用操作
- ◆ 案例

字典

- 字典：使用键值对（key: value）来存储数据，每一个键都对应一个值，通过键（key）可以快速找到对应的值（value）。
- 特点：键值对（key: value）存储、键（key）不能重复、可修改。
- 定义：

```
# 定义字典
```

```
字典名称 = {key: value, key:value, key:value ...}
```

```
# 定义空字典
```

```
字典名称 = {}
```

```
字典名称 = dict()
```

```
# 根据key获取value
```

```
值 = 字典名称[key]
```

```
# 定义字典
```

```
dict1 = {"王林": 675, "李慕婉": 608, "许立国": 478, ...}
```

```
# 定义空字典
```

```
dict2 = {}
```

```
dict3 = dict()
```

```
# 根据key获取value
```

```
score = dict1["李思"]
```

注意：字典（dict）中的value可以是任何类型的数据，而key不能为可变类型（如：不能为 列表list、集合set、字典dict）。

小结

1. 字典的存储形式及特点？

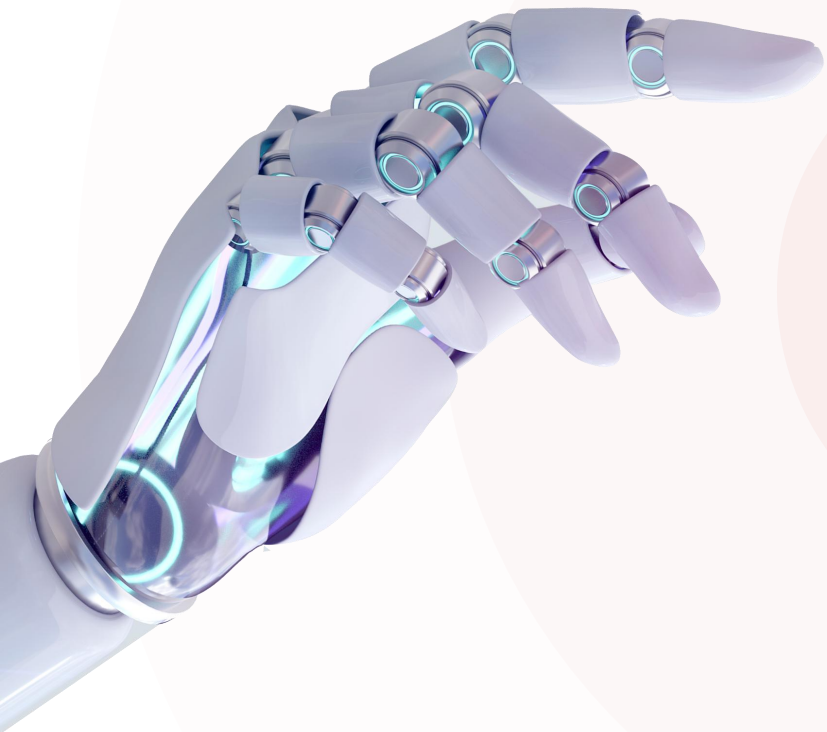
- 字典(dict)存储的是键值对(key:value)形式的数据
- 特点：键值对、key是唯一的、可修改

2. 字典定义的语法？

- 定义：dict1 = {"王林": 675, "李慕婉": 608, "许立国": 478, ...}
- 获取：v = dict1["王林"]

3. 字典的注意事项

- value可以是任意类型，而key必须是不可变类型（不能为list、set、dict）
- 字典内的key不允许重复，如果重复定义，后面的覆盖前面的
- 字典是没有索引下标的，不能根据索引获取值，只可以根据key获取value



5

字典(dict)

- ◆ 字典定义
- ◆ 常用操作
- ◆ 案例



常用操作

多一句没有，少一句不行，用更短的时间，教会更实用的技术

● 字典的增删改查操作方式如下：

```
dict1 = {"小智": 675, "李思": 608, "李琪": 478, "小黑": 545, "温韬": 429}
```

类型	操作	含义	样例
添加	字典名称[key] = value	往指定字典中添加key-value键值对	dict1["涛哥"] = 688
删除	字典名称.pop(key)	删除字典中指定的key, 并返回该key对应的value	score = dict1.pop("涛哥")
	del 字典名称[key]	删除字典中指定的键值对	del dict1["涛哥"]
修改	字典名称[key] = value	修改字典中指定的key对应的值	dict1["小智"] = 658
查询	字典名称[key]	根据key获取value	dict1["涛哥"]
	字典名称.get(key)	根据key获取value	dict1.get("涛哥")
	字典名称.keys()	获取所有的key	dict1.keys()
	字典名称.values()	获取所有的value	dict1.values()
	字典名称.items()	获取所有的key-value键值对	dict1.items()

BRIEF 小结

1. 字典的常用操作？

- 添加：字典[key] = value （key不存在，就会执行新增）
- 删除：del 字典[key] / value = 字典.pop(key)
- 修改：字典[key] = value （key存在，就会执行修改）
- 查询：字典[key] / 字典.get(key) ; 字典.keys() / 字典.values() / 字典.items()

2. 字典的遍历？

- 字典支持for循环遍历
- 通过keys()、values()、items() 进行循环遍历

5

字典(dict)

- ◆ 字典定义
- ◆ 常用操作
- ◆ 案例



案例

完成如下需求



- 开发一个购物车管理系统，实现商品信息的添加、修改、删除、查询功能。系统使用字典结构存储商品数据，通过控制台菜单与用户交互。具体功能如下：
 1. 添加购物车：用户根据提示录入商品名称、以及该商品的价格、数量，保存该商品信息到购物车。
 2. 修改购物车：要求用户输入要修改的购物车商品名称，然后再提示输入该商品的价格、数量，输入完成后修改该商品信息。
 3. 删除购物车：要求用户输入要删除的购物车名称，根据名称删除购物车中的商品。
 4. 查询购物车：将购物车中的商品信息展示出来，格式为："商品名称: xxx, 商品价格: x"

5. 退出购物车

购物车列表数据

```
shopping_cart = {"name": "Meta80", "price": 6999, "num": 2}
```

运行

15. 数据容器-dict x

D:\Python-Project\py_project01\.venv

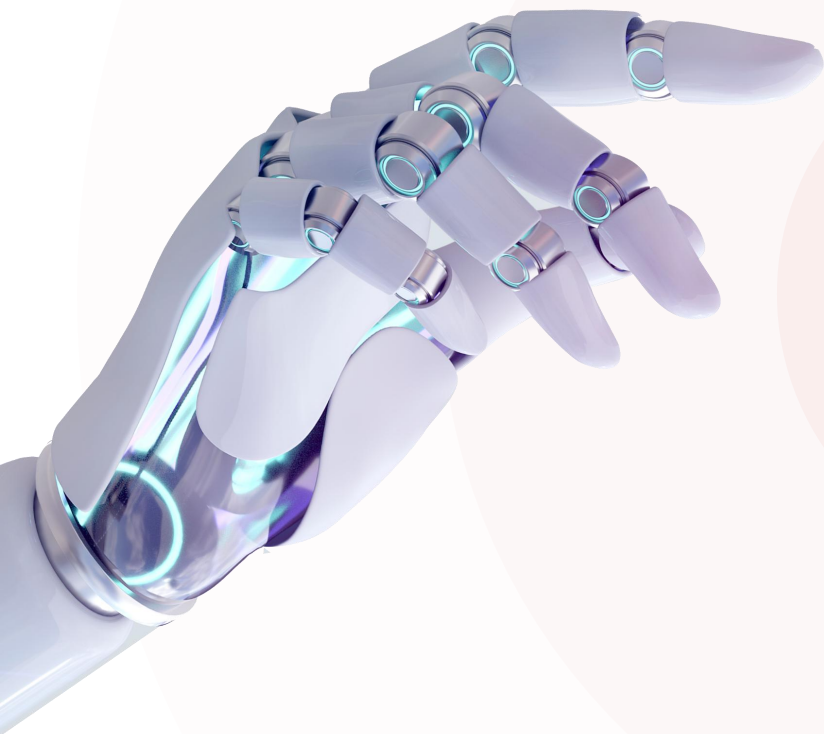
欢迎使用购物车管理系统！

购物车系统

```
#          1. 添加购物车          #
#          2. 修改购物车          #
#          3. 删除购物车          #
#          4. 查询购物车          #
#          5. 退出购物车          #
```

#####

请选择要执行的操作(1-5):



CONTENT 目录

- ◆ 列表 (list)
- ◆ 字符串 (str)
- ◆ 元组 (tuple)
- ◆ 集合 (set)
- ◆ 字典 (dict)
- ◆ 总结



数据容器总结与对比

特性	字符串 (str)	列表 (list)	元组 (tuple)	集合 (set)	字典 (dict)
有序性	有序	有序	有序	无序	有序(3.7+)
重复元素	允许	允许	允许	不允许	key不允许
可变性	不可变	可变	不可变	可变	可变
索引访问	支持	支持	支持	不支持	不支持
切片操作	支持	支持	支持	不支持	不支持
使用场景	文本处理	有序可重复数据集合	固定数据记录	去重数据集合	键值对

练习

基于现有知识开发一个教务管理系统



开发一个教务管理系统，在该系统中可以维护和管理学员的成绩信息，具体需求如下：

```
##### 【菜单】 #####
# 1. 添加学生信息    2. 修改学生信息    3. 删除学生信息    4. 查询学生信息    5. 列出所有学生    6. 统计班级成绩    7. 退出系统    #
#####

3. 删 请选择要执行的操作(1-7): 1

      请输入学生姓名: 李思

4. 查 请输入语文成绩: 92

5. 列 请输入数学成绩: 80

      请输入英语成绩: 78

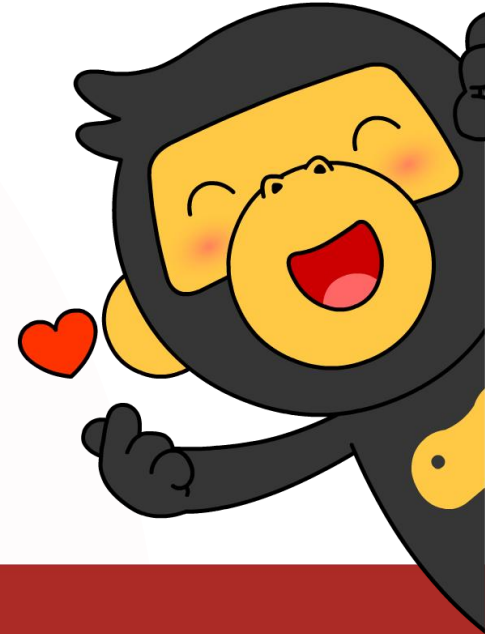
6. 统 添加 【李思】 成功

学

7. 退 ##### 【菜单】 #####
# 1. 添加学生信息    2. 修改学生信息    3. 删除学生信息    4. 查询学生信息    5. 列出所有学生    6. 统计班级成绩    7. 退出系统    #
#####
```

学员信息。

高分和最低分的



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌